403 Forbidden Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ HTTP 403 Forbidden — Access-Control Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Test only in-scope systems. Do not brute-force large path lists, probe unrelated hosts, or attempt to reach administrative data without explicit authorization.*

■ Executive Summary

A `403 Forbidden` response indicates that a server understood a request but refused access. Security testing should determine whether the refusal is consistently enforced across routes, HTTP methods, proxy layers, and identity contexts—not simply whether a response can be changed by altering request metadata.

■ Core Testing Methodology

1. **Confirm scope and baseline.** Record the URL, method, role, status, and expected access policy.
2. **Compare authorized roles.** Use test accounts with different permissions to verify that protected resources remain restricted.
3. **Review route normalization.** In a staging environment, check whether the front end and application interpret paths consistently, including encoded or normalized forms.
4. **Review method handling.** Confirm that GET, POST, and any supported alternate methods enforce the same authorization policy for equivalent actions.
5. **Review proxy/header trust.** Determine whether forwarding or host-related headers are accepted only from trusted infrastructure.
6. **Check canonicalization consistency.** Verify that the reverse proxy, web server, and application agree on the normalized path and resource.
7. **Review discovery controls.** Use a small, approved route list or documented endpoints; do not run uncontrolled brute-force discovery.
8. **Inspect errors safely.** Confirm that denied responses do not leak stack traces, internal paths, or sensitive metadata.
9. **Validate the actual impact.** A changed status code alone is not a vulnerability; confirm whether protected data or an action became available using a test object.
10. **Report and retest.** Include the access policy, test role, request variation category, and redacted evidence.

■ High-Value Bug Bounty Tips & Edge Cases

- **Route discovery:** The source mentions fuzzing and wordlists. Prefer a bounded list of in-scope, likely routes and respect program limits.
- **Header variations:** The PDF lists many proxy and host-related header names. These should never be trusted from arbitrary clients for authorization decisions.
- **Loopback values:** The source shows loopback-style header values. Do not assume such a value is trustworthy; trust must be based on the network boundary and proxy configuration.
- **URL/path variations:** The source contains a large collection of encoded and alternate path forms. Treat these as canonicalization test categories, not a payload list to spray against production.
- **Status-code changes:** A `200` response is not proof of a bypass. Check the response body, session state, and whether protected functionality is actually accessible.
- **Source limitation:** The PDF is primarily a list of tools, header names, and path variations; it does not establish that a particular target is vulnerable.

■■ Educational Checklists & Safe Examples

```
GET /[IN-SCOPE-PROTECTED-RESOURCE] HTTP/1.1
Host: example.com
Authorization: Bearer [CONTROLLED-TEST-TOKEN]
```

Test with an account that is not authorized for the resource.

```
Expected: 403 or another documented safe denial
Observed status: [Status]
Sensitive content returned: [Yes/No]
Action performed: [Yes/No]
Evidence: [Redacted]
```

- [ ] Authorization is enforced by the application, not by obscurity.
- [ ] Proxy headers are accepted only from trusted proxies.
- [ ] Path normalization is consistent across layers.
- [ ] Alternate methods do not bypass access policy.
- [ ] Denied responses do not leak sensitive data.
- [ ] Testing stays within the defined route and request budget.

■■ Mitigation & Remediation

- Enforce authorization at the application/resource layer for every request.
- Configure reverse proxies to strip untrusted forwarding headers and set trusted values themselves.
- Normalize paths once and consistently; reject ambiguous encodings.
- Keep proxy, web server, router, and framework access rules aligned.
- Avoid relying on IP headers, host headers, hidden URLs, or status codes as authorization.
- Log repeated denied access attempts and safely rate-limit abusive discovery.
- Add regression tests for roles, methods, path normalization, and proxy boundaries.

■ Source Notes

Based on the supplied 11-page *bypass 403.pdf*. Page 1 mentions tools, path discovery, and header names; pages 2 and 4 show header-value examples; later pages contain extensive URL/path variants. Because the document is largely a payload catalogue, this rewrite groups the material into safe testing categories rather than reproducing a bulk bypass list.

Educational use only. Test ethically and stay within scope.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.